

# **DS203**

# **Programming for Data Science**

## **Exercise 7**

## **Project**

### **GROUP 9**

**21D170016: Garv Gupta**

**21D170046: Vishesh Kamani**

**21D171001: Akash Sansagu Palaniswami**

**21D180011: Faiz Nawaz**

# Problem Statement

**An architecture firm has digitized its building layouts into 640x480 bitmap images to enhance productivity and adapt to customer needs. They aim to use data science and machine learning to categorize designs into families based on structural shapes, deepening design insights and standardizing processes through templates.**

# Problem Statement

**Classify the complexity of their building layouts. Perform an analysis to develop criteria for categorizing layouts into Low, Medium, and High Complexity. Utilize analytical techniques to establish these criteria and classify the layouts, enhancing design precision and efficiency.**

# Problem Statement

**Accelerate the design process by retrieving relevant prior layouts based on parameters such as the dimensions of the tight-fitting box, layout area, and permissible complexity. By specifying these parameters, we should predict and retrieve designs from the most relevant families.**

# Our Approach

- Use OpenCV to extract key features from layout images. Apply clustering algorithms to classify images into families.
- Weights to features and calculate complexity scores using these weights. Define thresholds for low, medium, and high complexity to categorize designs effectively.
- Train a machine learning model using tight-fitting box, layout area, and complexity for classification and use the model to predict relevant design families

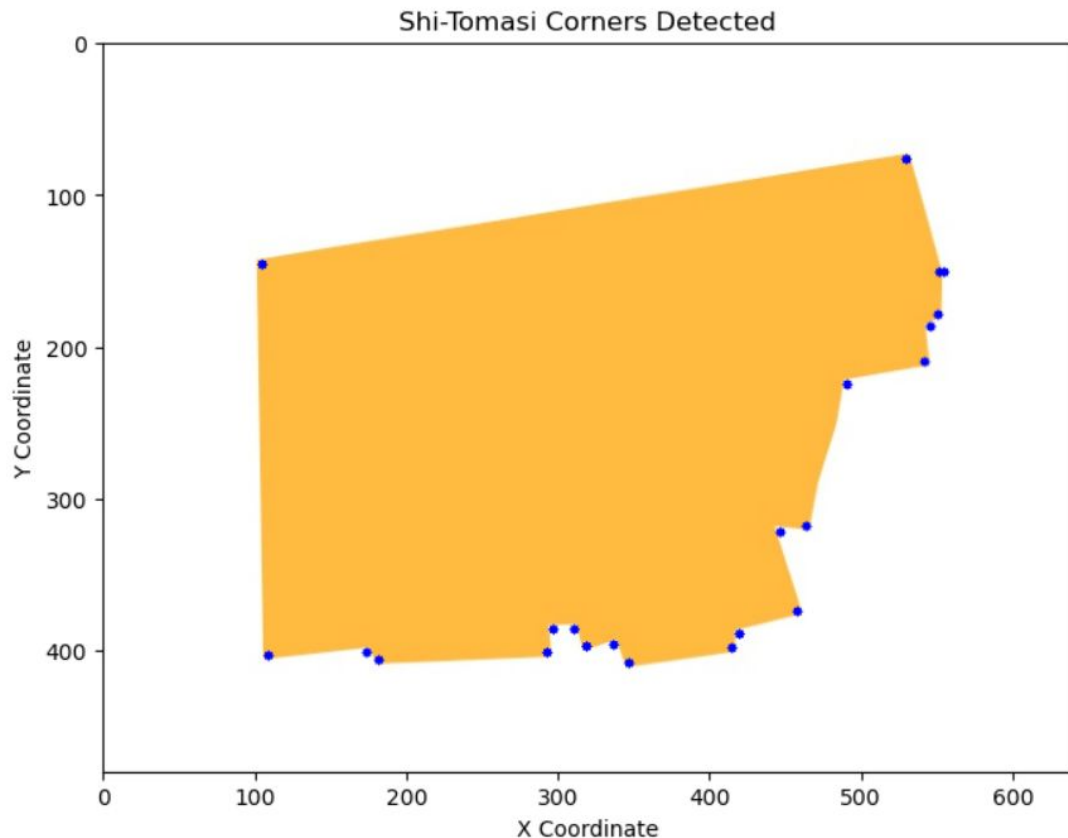
# Features Deployed For Clustering

- ❖ Variance of Internal Angles
- ❖ Variance of Side Length
- ❖ Number of Corners
- ❖ Number of Concave Corners
- ❖ Average Distance from Centroid
- ❖ Variance of Distance from Centroid

# Counting Number of Corners

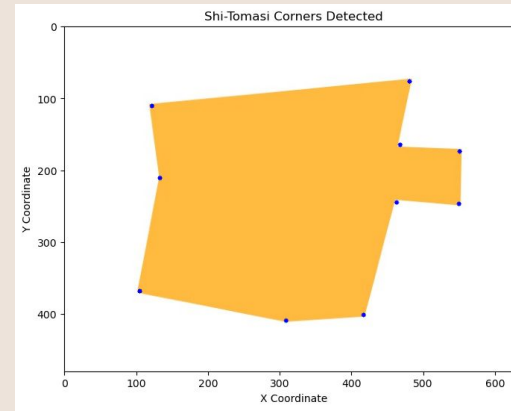
We are using the Shi-Tomasi Corner Detector to get the corners. It has been implemented using `goodFeaturesToTrack` by OpenCV

Example: image 0783 gives us **22 corners**



# Other Features

- The corners are detected using Shi-Tomasi
- The interior angle and side length information is extracted using OpenCV to make a contour
- Canny edge detection is used to get the centroid data



| <u>Features for Image 0284</u> | <u>Value</u> |
|--------------------------------|--------------|
| Std of Angles                  | 71.79        |
| Std of Sides                   | 83.13        |
| No. of Corners                 | 10           |
| No. of Concave Corners         | 3            |
| Avg Distance from Centroid     | 203.58       |
| Std Distance from Centroid     | 35.68        |

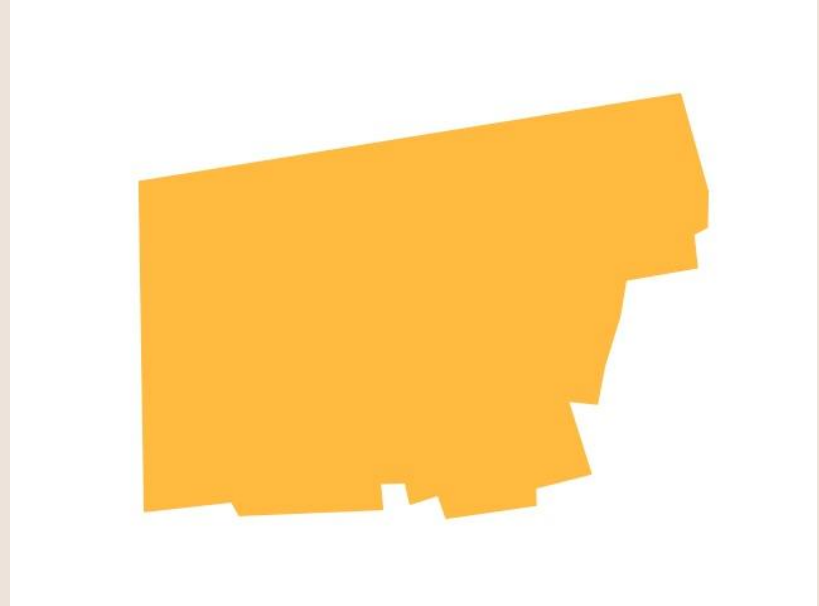


# How are all the features extracted?

- ❑ Number of corners in image are ascertained by approximating the contour using approxPolyDP and finding its length
- ❑ For the standard deviation of angle we take the coordinates of 3 points and find the angle using the cross product of the vectors while also adjusting in case of concave angles.



- ❏ Distance of each vertices is calculated from the centroid and its mean and standard deviation is taken for comparison
- ❏ Using epsilon value approximation for arc length to figure out the standard deviation in angles and number of concave corners (when angle is greater than  $180^\circ$ )
- ❏ Contour area is calculated directly using in-built function `contourArea()`



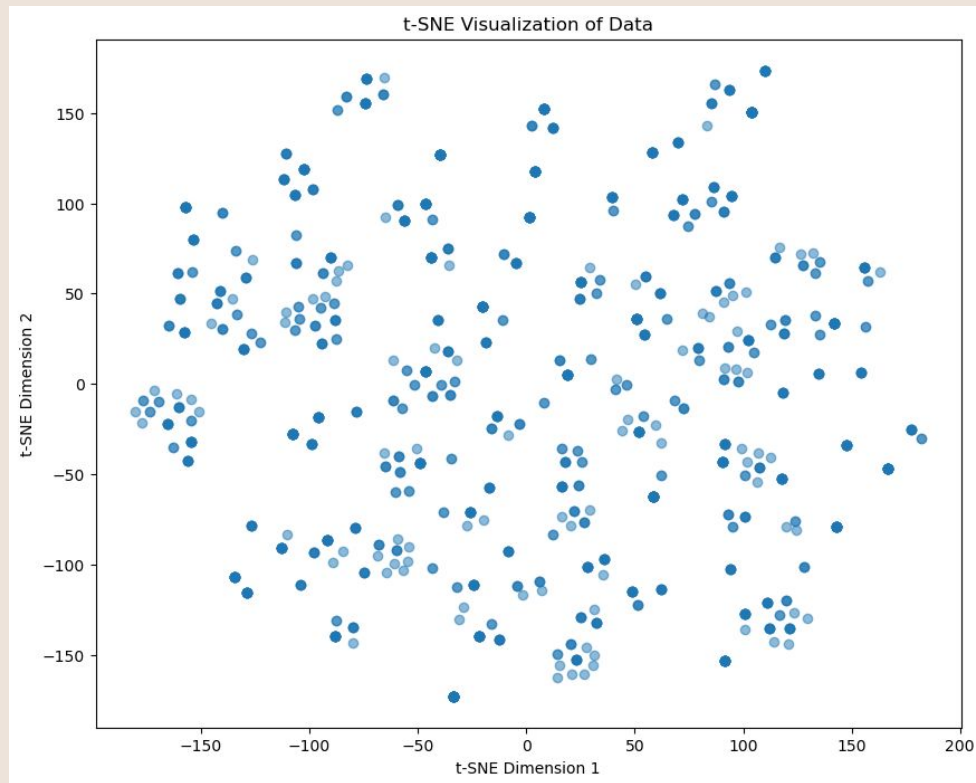
Contour Area comes out to be: **115353.5**

# Classification into Families

- ❖ **t-SNE Dimensionality Reduction**
- ❖ **K-Means Clustering**
- ❖ **Hierarchical Clustering**

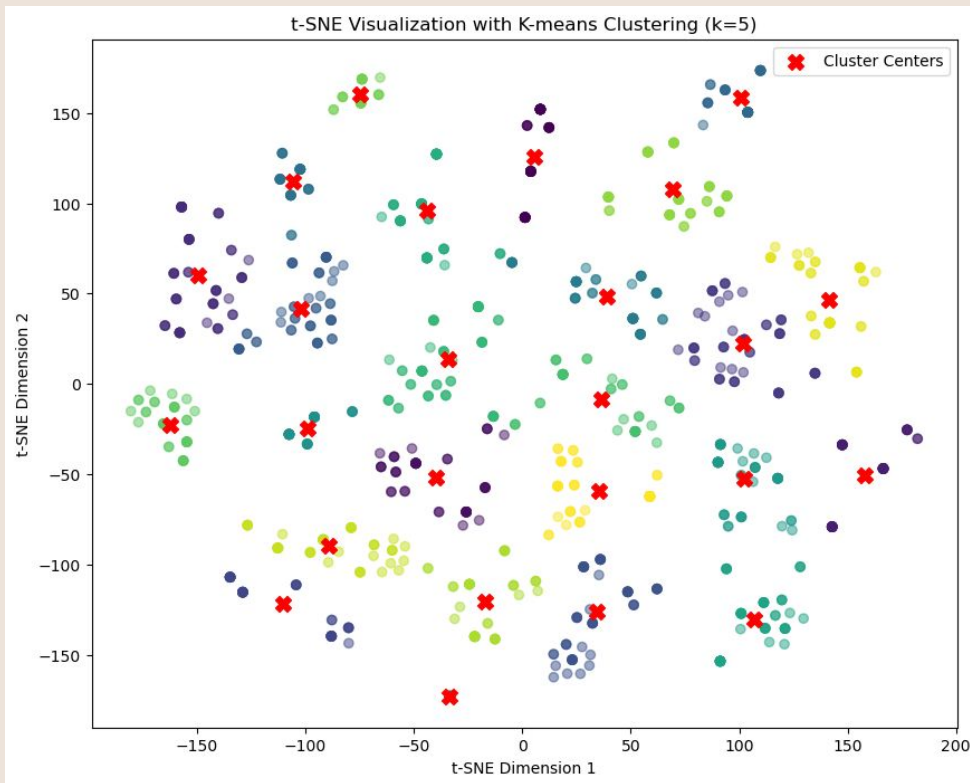
# t-SNE Dimensionality Reduction

We used t-SNE to reduce dimensions and produce a 2D visualization. We see that there are indeed small clusters which could represent a family of designs with similar features.



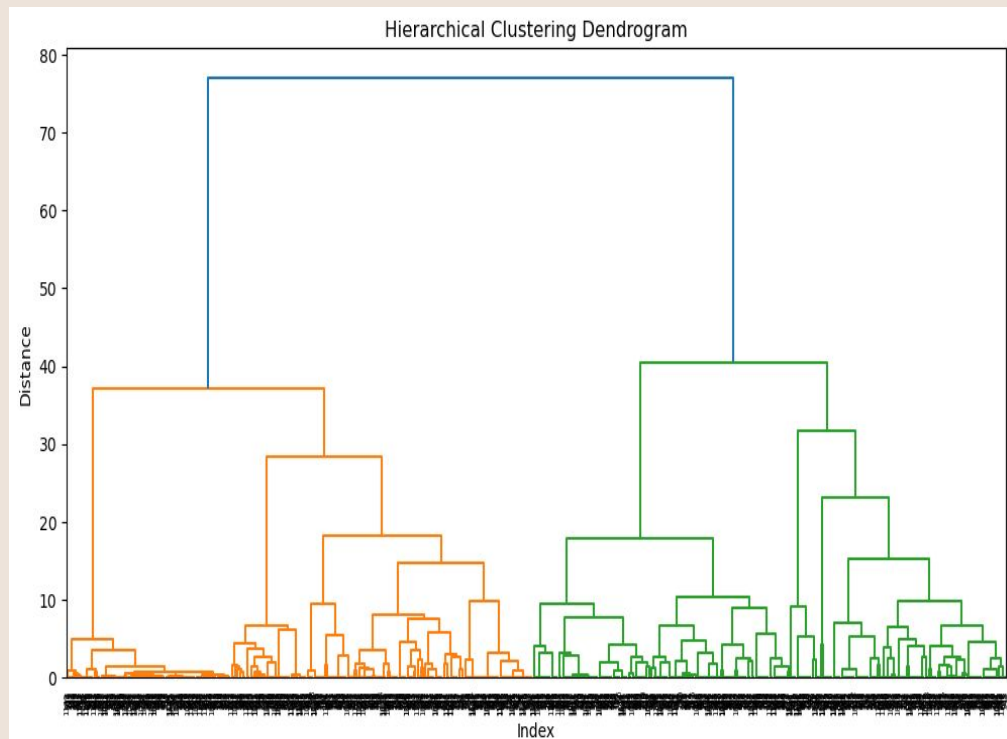
# Using K-means

We then applied K-means clustering to segment the data into 25 distinct clusters. This categorizes designs into families, revealing patterns and similarities within the dataset. The features for clustering are the t-SNE dimensions



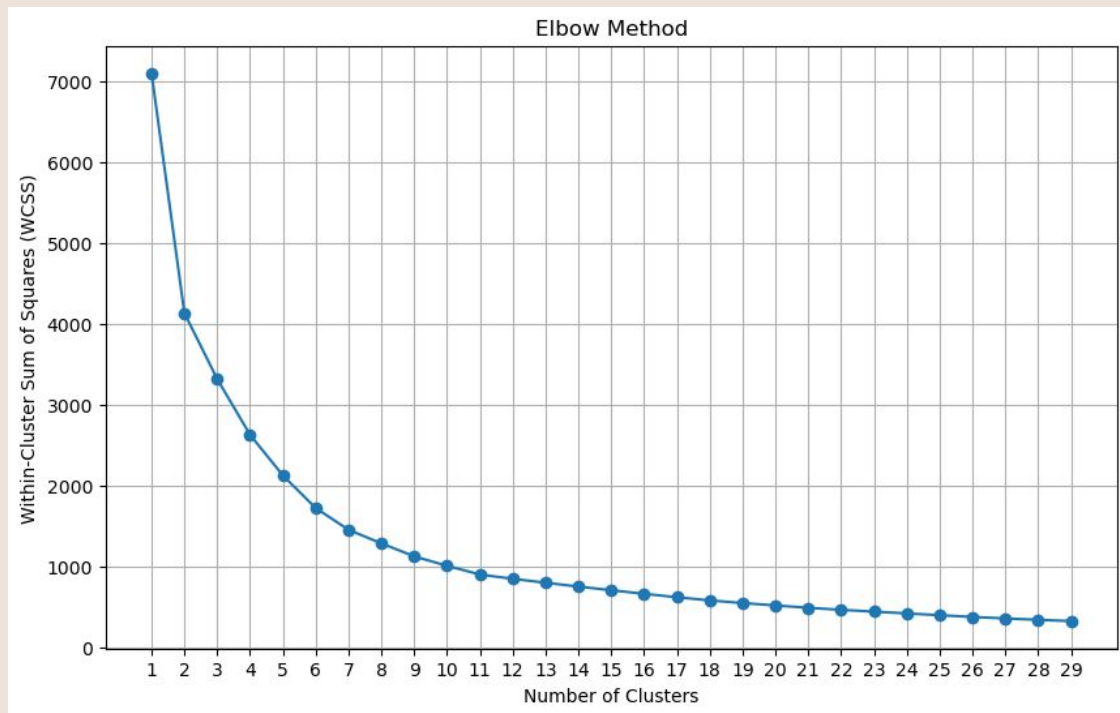
# Using Hierarchical Clustering

For our second approach, we employed hierarchical clustering using all the original features. This method organized the data into a tree-like structure, allowing us to see a detailed breakdown of the dataset into progressively smaller groups based on similarity.



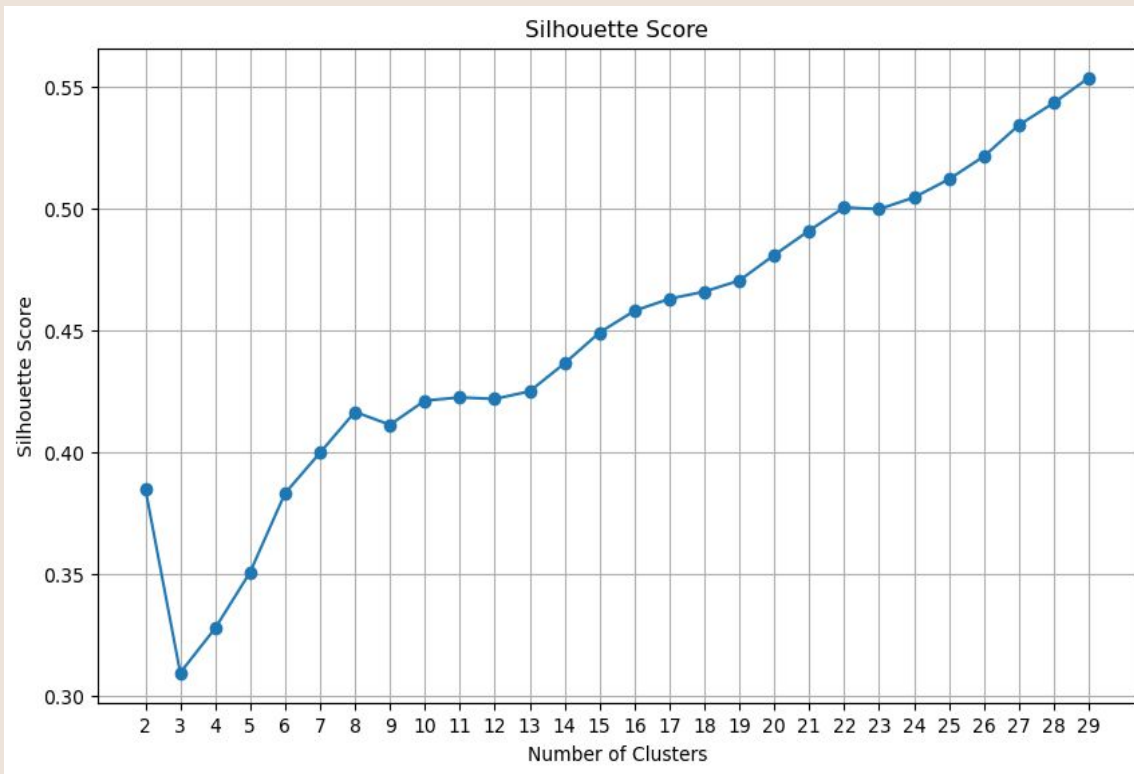
# Finalising the number of clusters

- Using **Elbow Method** with an assumption of 30 maximum clusters, calculating the within-cluster sum of squares for different number of clusters.
- After value of **n=26** there is no change so we assume that 26 can be taken as an approximate number of clusters for our designs



# Finalising the number of clusters

- Similarly using another metric Silhouette Score, we get an appropriate number of clusters.
- In both the above methods we have scaled data and 'maxclust' criteria





# Finalising the number of clusters

**Using a higher number of clusters ensures high intra-cluster similarity, which facilitates the use of these clusters as cohesive families in subsequent analyses. This approach allows us to address broader problems by leveraging these well-defined families rather than individual images.**

# Complexity Classification

We decided to group the 25 clusters into, different complexities assuming design families have similar complexity

Features used for classifying the 25 clusters:

- ❖ Number of corners
- ❖ Number of concave corners
- ❖ Standard deviation of the distance of vertices from centroid

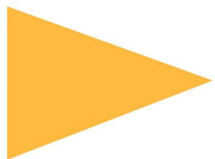
# Complexity Classification

We calculated the complexity score for each design family by taking a weighted sum of the average features. Based on our analysis, we then set appropriate thresholds to categorize the families into different levels of complexity keeping roughly equal number of images in each complexity.

No. of images with:

- Low complexity: 348
- Medium Complexity: 496
- High Complexity: 339

## Low Complexity



## Medium Complexity

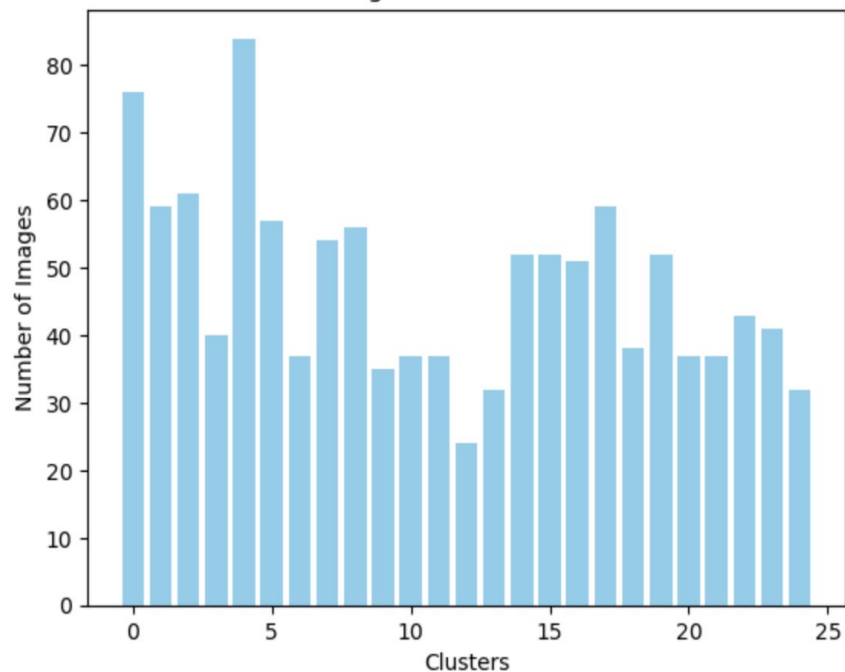


## High Complexity

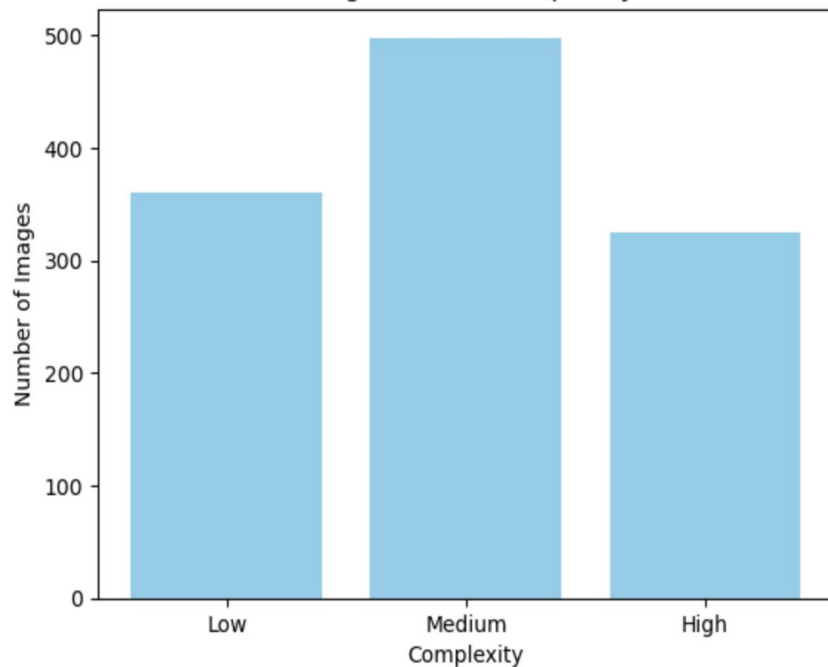


# Some Results from the Analysis

Images in Each Cluster



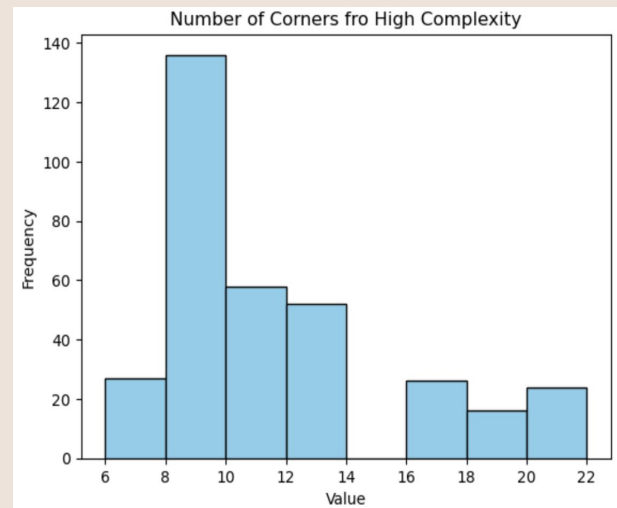
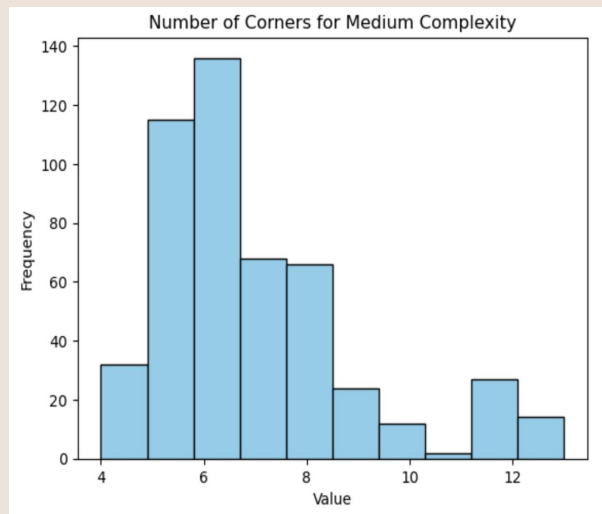
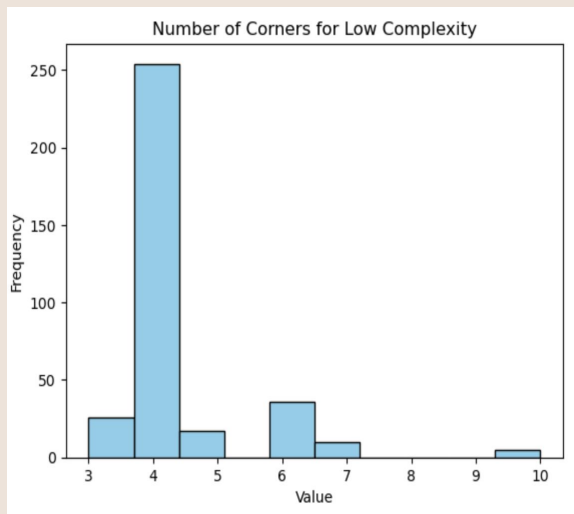
Images in Each Complexity



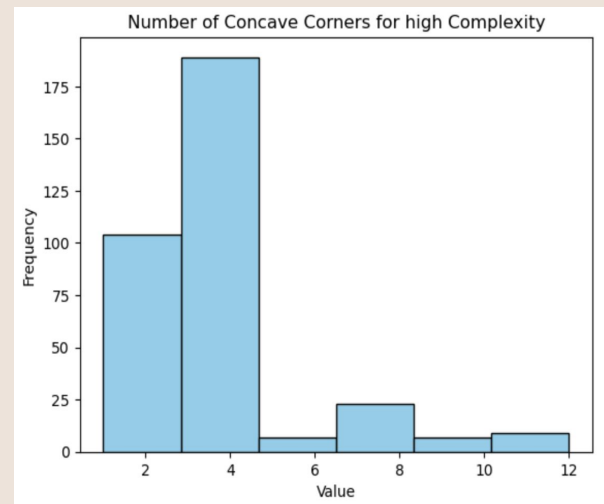
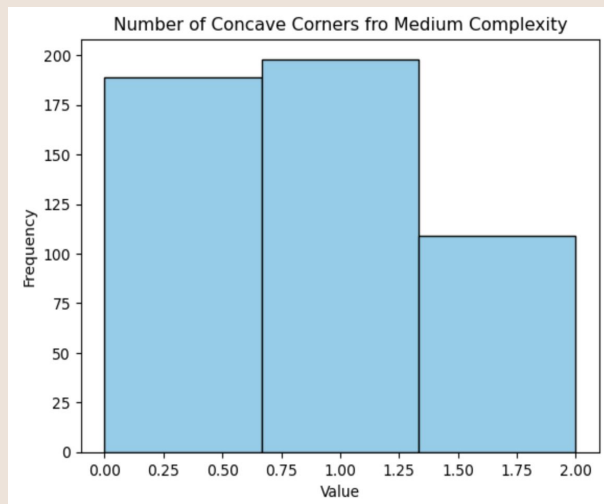
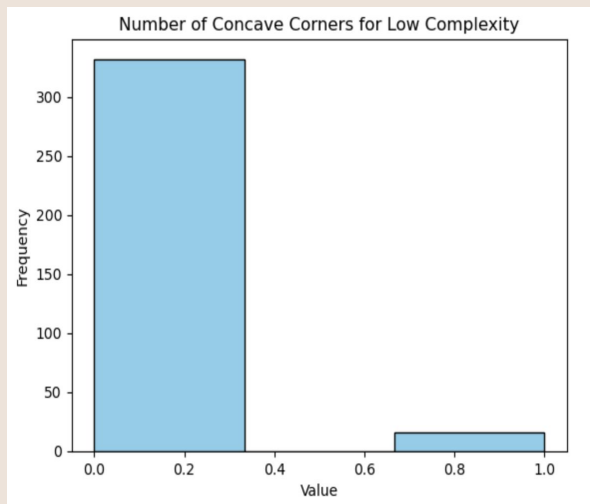
# Metric Values for each complexity

| Metrics                                              | Low Complexity(0) | Medium Complexity(1) | High Complexity(2) |
|------------------------------------------------------|-------------------|----------------------|--------------------|
| Average Number of corners                            | 4.353             | 6.828                | 11.41              |
| Average number of concave corners                    | 0.046             | 0.8387               | 3.539              |
| Average Standard Deviation of angles                 | 10.026            | 42.177               | 61.553             |
| Average Standard Deviation of side length            | 88.683            | 111.08               | 91.09              |
| Average Standard Deviation of distance from centroid | 7.906             | 34.123               | 45.067             |

# Number of corners distribution for each complexity



# Number of Concave corners distribution for each complexity





# Prediction and Retrieval of Relevant Prior Layouts

- ❖ For this problem, we would want to predict the family which has the highest similarity to the parameters provided
- ❖ We trained machine learning classifiers on parameters like dimensions of the tight-fitting box, layout area, and complexity with the target variable being the cluster label for the image
- ❖ We then made a retrieval system which would display some of designs from the recommend family

# Prediction and Retrieval of Relevant Prior Layouts

- ❖ We have used OpenCV function `minAreaRect` to get the dimensions of the Tight-Fitting Box
- ❖ We tested three classifiers: Random Forest, Support Vector Machine, and XGBoost, to evaluate their performance on our dataset

# Classification report for each Classifier method

## Random Forest Classifier

| Metric           | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| accuracy         | -         | -      | 0.97     | 237     |
| Macro average    | 0.98      | 0.97   | 0.97     | 237     |
| Weighted average | 0.98      | 0.97   | 0.97     | 237     |

## Support Vector Machine Classifier

| Metric           | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| accuracy         | -         | -      | 0.15     | 237     |
| Macro average    | 0.08      | 0.12   | 0.07     | 237     |
| Weighted average | 0.08      | 0.15   | 0.08     | 237     |

Based on the final values of performance parameters, we can say that XGBoost and Random Forest classifiers give us the best results while support vector machine isn't able to give good prediction.

## XGBoost Classifier

| Metric           | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| accuracy         | -         | -      | 0.97     | 237     |
| Macro average    | 0.97      | 0.97   | 0.97     | 237     |
| Weighted average | 0.98      | 0.97   | 0.97     | 237     |

- Precision: Accuracy of positive predictions.
- Recall: Coverage of actual positive cases.
- F1-score: Harmonic mean of precision and recall.
- Support: Number of true instances for each class.

# Summary

**Classification of Designs:** We grouped designs into families based on key features, ensuring each family had similar designs.

**Complexity Categorization:** Assigned complexity levels to these families.

**Recommendation System:** Developed a machine learning-based system to match new designs with the best-fitting family templates.

**THANK YOU!**